

Урок 8.3 — Практика (проверенный гайд для демонстрации)

Этот гайд проверен на текущем проекте LaravelExample и заточен под демонстрацию студентам без сбоев.

Цель практики:

- показать поток логов в Docker;
- показать структурированный лог ошибки платежа;
- показать **X-Correlation-ID** в запросе, ответе и логе;
- пройти mini incident-flow на базе логов.

0) Добавление Context в проект (перед практикой)

Context в Laravel — фасад для хранения контекста текущего выполнения (request / job / command), чтобы единообразно прокидывать метаданные в логи и error tracking.

Что добавлено в проект:

- в **AssignCorrelationId** теперь используется **Illuminate\Support\Facades\Context**;
- в **Context** пишутся базовые поля запроса: **correlation_id**, **endpoint**, **request_method**, **request_path**, **user_id**;
- **Log::shareContext(...)** получает данные из **Context::all()**;
- **PaymentErrorReporter** объединяет доменные поля ошибки с **Context::all()** и передает их в лог/Sentry.

Зачем это нужно:

- один источник правды для метаданных диагностики;
- меньше дублирования (**correlation_id** не нужно вычислять в каждом сервисе);
- проще связывать API-ответ, логи и Sentry-события.

Где хранятся данные **Context**:

- это in-memory хранилище в рамках текущего процесса выполнения;
- для HTTP-запроса контекст живет в памяти текущего запроса;
- для queue/job контекст может быть сериализован/восстановлен механизмом Laravel, если передавать его через pipeline выполнения задач.

Базовые возможности:

- **Context::add('correlation_id', \$id)** — добавить значение;
- **Context::get('correlation_id')** — получить значение;
- **Context::all()** — получить весь контекст одним массивом.

0.1 До внедрения Context (кейс с ошибкой)

Без **Context** типовой поток выглядит так:

- middleware выставляет только header и частичный log-context;
- сервисы отдельно достают `X-Correlation-ID` из запроса;
- при росте числа полей (`tenant_id`, `request_id`, `user_id`, `endpoint`) растёт дублирование.

Пример:

- в контроллере берётся `X-Correlation-ID` из `Request`;
- в `Log::error(...)` вручную передается набор полей;
- в Sentry `extra` добавляются поля отдельными вызовами.

0.2 После внедрения Context (текущий проект)

Теперь поток централизован:

- middleware один раз наполняет `Context`;
- контроллер/сервисы читают `Context::get(...)` при необходимости;
- логирование ошибок автоматически получает общий контекст через `Context::all()`.

Практический эффект:

- ошибки платежей содержат и доменные поля (`payment_id`, `gateway_code`), и request-метаданные (`correlation_id`, `endpoint`);
- меньше риска потерять диагностические поля при новых точках логирования;
- проще эволюционировать observability-подход без широких рефакторингов.

1) Что важно знать до старта

В проекте по умолчанию в `.env`:

- `LOG_CHANNEL=stack`
- `LOG_STACK=single,sentry_logs`

То есть Laravel пишет в файл (`storage/logs/laravel.log`) + Sentry logs, а не в stdout контейнера.

Для урока 8.3, чтобы логи были видны через `docker compose logs`, временно переключаем канал на `stderr`.

2) Поднять и проверить окружение

```
docker compose up -d
docker compose ps
docker compose exec php php artisan test
```

Ожидаемо:

- все сервисы Up (`php`, `mysql`, `redis`, `queue-worker`, `adminer`);
- тесты проходят (в текущем состоянии: `26 passed`).

3) Включить вывод Laravel-логов в stdout/stderr контейнера (для демо)

2.1. Сделать backup `.env`

```
docker compose exec php sh -lc 'cp .env .env.lesson83.bak'
```

2.2. Временно переключить логирование

```
docker compose exec php sh -lc "sed -i  
's/^LOG_CHANNEL=.* /LOG_CHANNEL=stderr/' .env && sed -i  
's/^LOG_STACK=.* /LOG_STACK=stderr,sentry_logs/' .env"  
docker compose restart php
```

После этого ошибки приложения будут видны в:

```
docker compose logs -f php
```

4) Демо 1 — Correlation ID в ответе API

Отправляем тестовый запрос с заранее заданным `X-Correlation-ID`:

```
curl -i -X POST http://localhost/api/v1/payments/demo-fail \  
-H "Accept: application/json" \  
-H "X-Correlation-ID: lesson83-demo-001"
```

Проверяем в ответе:

- статус `202 Accepted`;
- заголовок `X-Correlation-ID: lesson83-demo-001`;
- JSON с `correlation_id: lesson83-demo-001`.

5) Демо 2 — Структурированный лог ошибки в контейнерных логах

Сразу после запроса выше (в течение 5-10 секунд):

```
docker compose logs --since=5m php
```

Если всё равно пусто, используйте стабильный вариант:

```
docker compose logs -f php
```

и в другом терминале повторно триггерните:

```
curl -i -X POST http://localhost/api/v1/payments/demo-fail \
-H "Accept: application/json" \
-H "X-Correlation-ID: lesson83-demo-002"
```

Ожидаем строку вида:

```
[2026-...] local.ERROR: Payment processing failed at gateway level
{"correlation_id":"lesson83-demo-001","payment_id":999999,"user_id":0,"gateway_code":"DEMO_FAIL"}
```

Что это доказывает для урока:

- есть структурированный JSON-контекст;
- `correlation_id` сквозной (из запроса в лог);
- событие легко искать и фильтровать в лог-агрегации.

6) Поднять локальную лог-агрегацию (Loki + Promtail + Grafana)

Ниже самый практичный вариант для live-демо: не “концептуально”, а реально поднять агрегатор и показать поиск логов по полям.

6.1. Добавить override compose-файл

Создайте `docker-compose.logging.yml` в корне проекта:

```
services:
  loki:
    image: grafana/loki:3.0.0
    container_name: lesson83_loki
    command: -config.file=/etc/loki/local-config.yaml
    ports:
      - "3100:3100"

  promtail:
    image: grafana/promtail:3.0.0
    container_name: lesson83_promtail
    command: -config.file=/etc/promtail/promtail.yaml
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - ./docker/promtail/promtail.yaml:/etc/promtail/promtail.yaml:ro
    depends_on:
      - loki

  grafana:
    image: grafana/grafana:11.1.0
    container_name: lesson83_grafana
    environment:
      GF_SECURITY_ADMIN_USER: admin
```

```
GF_SECURITY_ADMIN_PASSWORD: admin
ports:
  - "3000:3000"
depends_on:
  - loki
```

6.2. Додати конфіг promtail

6.1.1. Пояснення базових параметрів `docker-compose.logging.yml` (українською)

- `image` — готовий Docker-образ сервісу (`loki`, `promtail`, `grafana`).
- `container_name` — фіксоване ім'я контейнера для зручного дебагу.
- `command` — з яким конфігом стартує сервіс (наприклад, `-config.file=...`).
- `ports` — проброс портів на хост:
 - `3100:3100` для Loki API;
 - `3000:3000` для Grafana UI.
- `volumes`:
 - `/var/run/docker.sock:/var/run/docker.sock:ro` — дає `promtail` read-only доступ до метаданих Docker-контейнерів;
 - `./docker/promtail/promtail.yaml:/etc/promtail/promtail.yaml:ro` — підключає локальний конфіг Promtail.
- `environment` — змінні оточення контейнера (у прикладі логін/пароль Grafana).
- `depends_on` — порядок запуску (спочатку `loki`, потім `promtail/grafana`).

Для `promtail.yaml` важливо:

- `clients.url` — куди штовхати логи (`http://loki:3100/loki/api/v1/push`);
- `docker_sd_configs` — звідки читати список контейнерів;
- `relabel_configs` — які labels додавати (наприклад, `container`, `service`) для фільтрації в Grafana.

Создайте файл `docker/promtail/promtail.yaml`:

```
server:
  http_listen_port: 9080
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 5s
    relabel_configs:
```

```
- source_labels: ['__meta_docker_container_name']
  regex: '/(.*)'
  target_label: container
- source_labels:
  ['__meta_docker_container_label_com_docker_compose_service']
  target_label: service
- source_labels:
  ['__meta_docker_container_label_com_docker_compose_project']
  target_label: compose_project
```

6.3. Запуск стека агрегации

```
docker compose -f docker-compose.yml -f docker-compose.logging.yml up -d
docker compose -f docker-compose.yml -f docker-compose.logging.yml ps
```

Ожидаемо Up:

- loki
- promtail
- grafana
- базовые сервисы проекта (php, mysql, redis, ...).

6.4. Подключить Loki в Grafana

1. Откройте <http://localhost:3000> (логин/пароль: admin/admin).
2. Connections -> Data sources -> Add data source -> Loki.
3. URL: <http://loki:3100> (если добавляете из контейнера Grafana) или <http://localhost:3100> (если UI проверяет с хоста, в зависимости от режима).
4. Save & test.

6.5. Показать агрегацию на живых логах

Сгенерировать события:

```
curl -i -X POST http://localhost/api/v1/payments/demo-fail \
-H "Accept: application/json" \
-H "X-Correlation-ID: agg-demo-001"
```

В Grafana Explore (Loki) покажите запросы:

- все логи приложения:

```
{container="finance_app"}
```

- только ошибки:

```
{container="finance_app"} |= "ERROR"
```

- конкретный correlation id:

```
{container="finance_app"} |= "agg-demo-001"
```

- только payment failure:

```
{container="finance_app"} |= "Payment processing failed at gateway level"
```

Что проговорить студентам:

- это уже не **tail** на одном контейнере, а централизованный поиск;
- один и тот же запрос в UI можно сохранить и использовать в инциденте;
- correlation_id дает трассировку кейса за секунды.

6.6. Остановить локальную агрегацию после демо

```
docker compose -f docker-compose.yml -f docker-compose.logging.yml down -v  
--remove-orphans
```

—v удалит связанные docker volumes (данные Loki/Grafana), чтобы после урока не оставалось накопленных логов и состояния.

7) Демо 3 — Мини incident-flow (по мотивам слайда 13a)

Сгенерировать серию ошибок с разными correlation id:

```
for i in 1 2 3; do  
  curl -s -o /dev/null -X POST http://localhost/api/v1/payments/demo-fail  
  \  
    -H "Accept: application/json" \  
    -H "X-Correlation-ID: incident-lesson83-00${i}";  
done
```

Проверка «масштаба» в логах:

```
docker compose logs --since=5m php
```

На разборе со студентами:

- подтверждаем всплеск **ERROR**;
- берем один **correlation_id** и показываем трассировку кейса;
- обсуждаем временные действия (rollback/ограничение функции/коммуникация);
- фиксируем, какие поля обязательны для post-mortem.

8) Привязка к слайдам

- **Слайд 6а:** показываем, что приложение пишет в поток контейнера (**stderr**), откуда это легко забирает любой collector.
- **Слайд 9а:** разбираем структуру **message + контекст** на реальном логе.
- **Слайд 13а:** делаем мини-сценарий инцидента на серии одинаковых ошибок.

9) Частые проблемы и быстрое решение

1. Нет логов в **docker compose logs php**

- проверьте, что включили **LOG_CHANNEL=stderr**;
- после изменения **.env** обязательно **docker compose restart php**.

2. **curl** не получает **X-Correlation-ID**

- проверьте middleware **AssignCorrelationId** в API-стеке;
- убедитесь, что запрос идет на **http://localhost/api/v1/....**

3. Контейнеры не стартуют

- выполните **docker compose ps**;
- при необходимости: **docker compose up -d --build**.

10) Возврат к исходной конфигурации после урока

```
docker compose exec php sh -lc 'mv .env.lesson83.bak .env'
docker compose restart php
```

Это вернет стандартный режим проекта (**stack + single, sentry_logs**).

11) Короткий чек-лист перед занятием

- **docker compose ps** — все сервисы Up;
- **docker compose exec php php artisan test** — зеленый прогон;
- **curl /api/v1/payments/demo-fail** возвращает **202** и **X-Correlation-ID**;
- **docker compose logs php** показывает **Payment processing failed...** с JSON-контекстом и нужным **correlation_id**.